RELATÓRIO DA TAREFA 2.3

Revisar Material Disponibilizado pelo Professor

Validação da infraestrutura Docker, PostgreSQL e Apache Airflow

Disciplina	Projeto de Análise e Fluxo de Dados - OLAP e ETL
Projeto	AdventureWorks Data Warehouse
Fase	Fase 2 - Consolidação dos materiais reutilizáveis
Tarefa	2.3 - Revisar material disponibilizado pelo professor
Material avaliado	unisales.etl.20262-main/aula.etl.2
Status	Concluída
Data	23/10/2026

1. Objetivo

A atividade teve como objetivo revisar tecnicamente a base disponibilizada pelo professor e verificar se ela poderia ser utilizada como infraestrutura inicial do projeto individual. Antes de realizar qualquer adaptação, a base foi executada em sua configuração original para comprovar seu funcionamento.

2. Estrutura do material fornecido

O diretório analisado continha os seguintes componentes principais:

```
.env.example
.gitignore
docker-compose.yml
readme.md
dags/
```

O ambiente foi estruturado com Apache Airflow 3.3.1, PostgreSQL 16 e LocalExecutor, além de DAGs de exemplo e scripts SQL de apoio.

3. Preparação do ambiente

- Criação das pastas logs, plugins, config e data.
- Criação do arquivo .env a partir de .env.example.
- Geração da chave Fernet e dos segredos utilizados pelo Airflow.
- Validação dos serviços definidos no Docker Compose.

A validação do Compose apresentou os seis serviços esperados:

```
postgres
airflow-init
airflow-apiserver
airflow-scheduler
airflow-triggerer
airflow-dag-processor
```

4. Inicialização do Airflow

A inicialização foi realizada com o serviço airflow-init. O PostgreSQL tornou-se saudável e o processo de inicialização criou o usuário administrador do Airflow.

```
User "admin" created with role "Admin"
airflow-init-1 exited with code 0
```

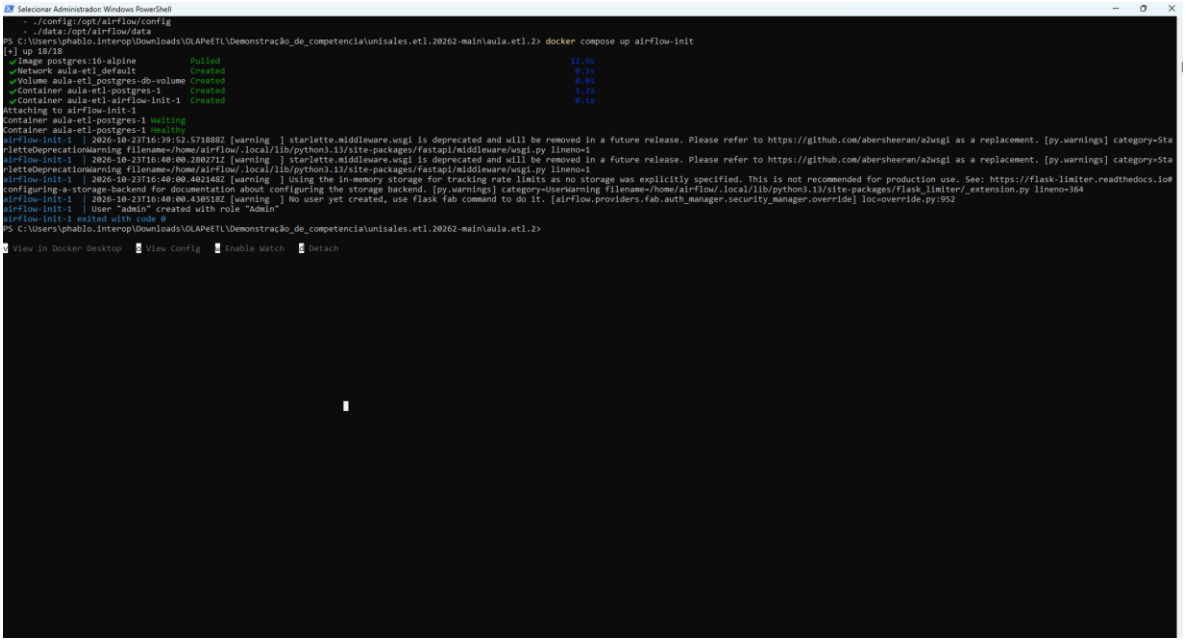


Figura 1 - Execução do airflow-init e criação do usuário administrador.

5. Subida dos serviços

Após a inicialização, os serviços foram iniciados em segundo plano. O PostgreSQL e o API Server atingiram estado saudável e os demais componentes do Airflow permaneceram ativos.

Serviço	Resultado observado
postgres	Healthy
airflow-apiserver	Healthy
airflow-scheduler	Ativo / posteriormente saudável na interface
airflow-dag-processor	Ativo
airflow-triggerer	Ativo
airflow-init	Finalizado com sucesso

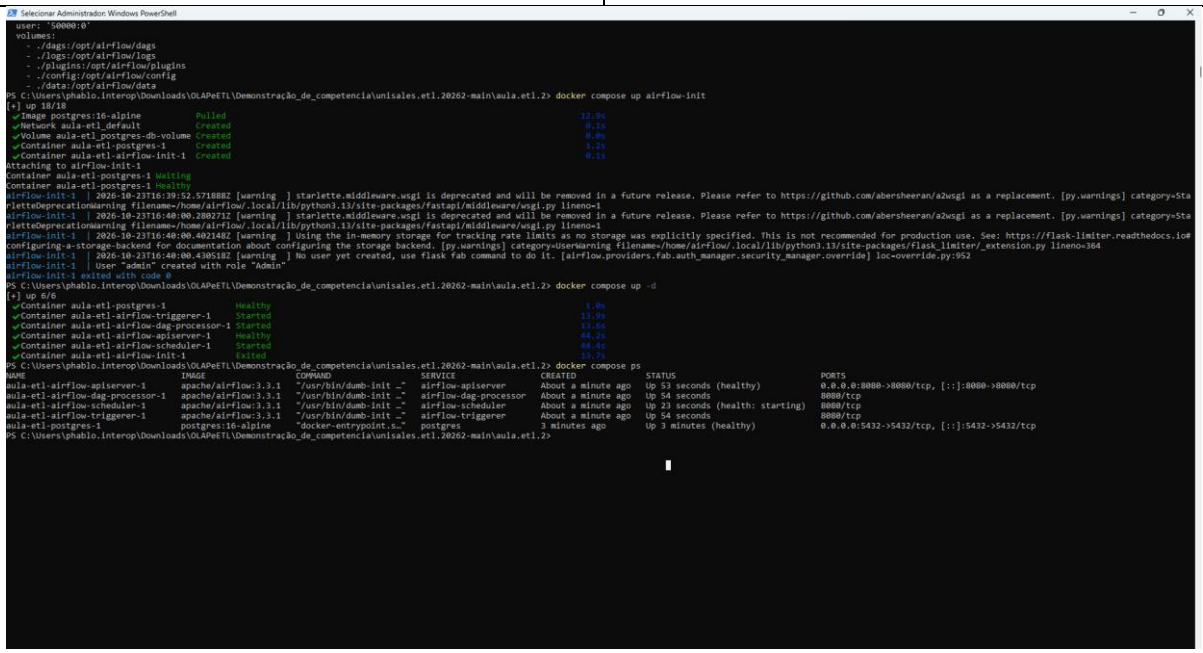


Figura 2 - Serviços do Docker Compose após a inicialização do ambiente.

6. Validação da interface web

A interface do Apache Airflow foi acessada pelo endereço local na porta 8080. O login com o usuário administrador criado na inicialização funcionou corretamente.

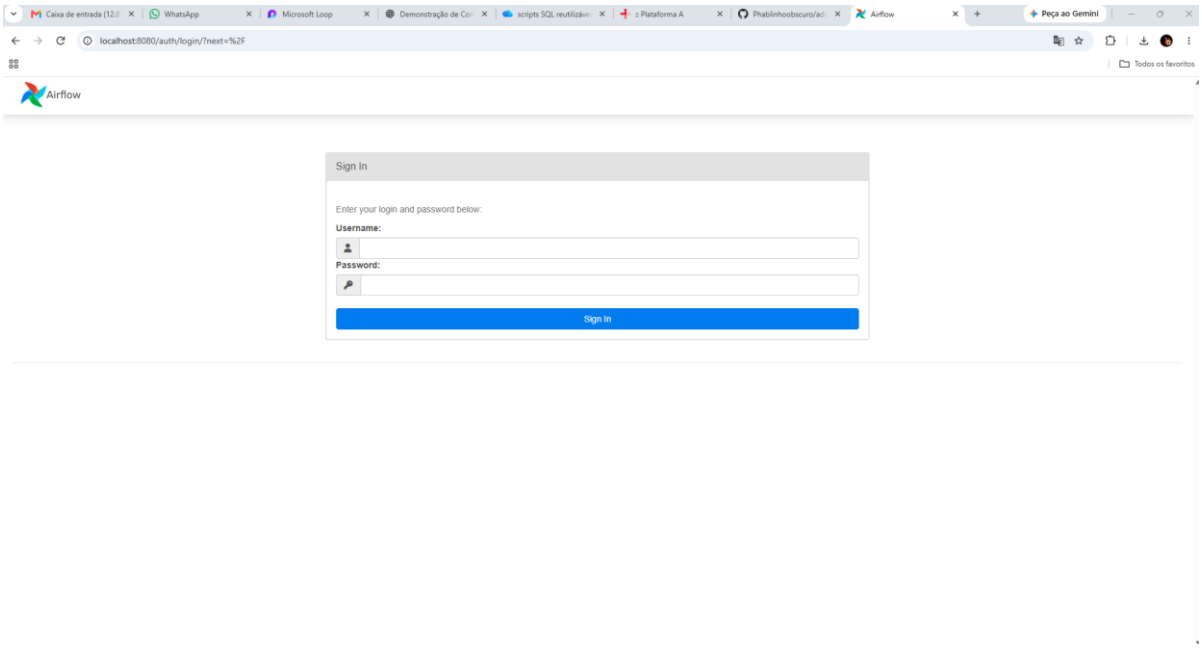


Figura 3 - Tela de autenticação do Apache Airflow.

Após o login, a página inicial apresentou os principais componentes do ambiente como saudáveis, incluindo Base de Dados de Metadados, Agendador, Disparador e Processador de DAG.

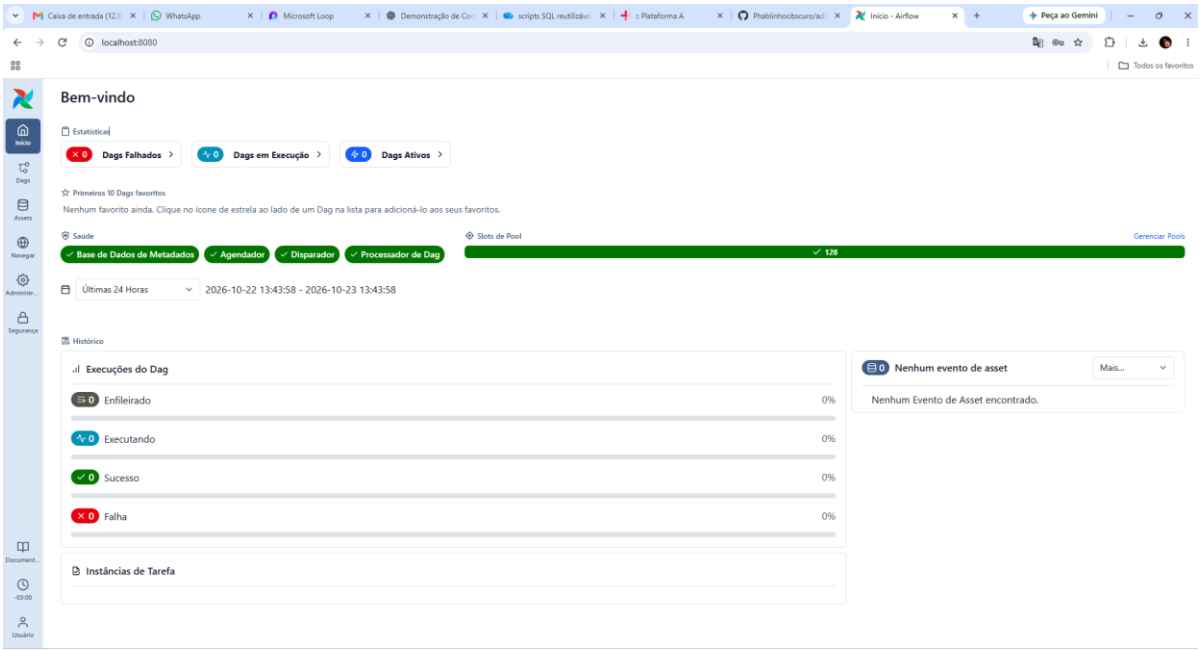


Figura 4 - Painel inicial do Airflow com os componentes do ambiente em funcionamento.

7. Execução de uma DAG de exemplo

Para validar não apenas a interface, mas também a execução real de um pipeline, foi utilizada a DAG `etl_exemplo_persistencia_local`, por não depender de conexões externas.

O fluxo executado foi:

```
extrair
   |
   +------------+
   |            |
   v            v
transformar  transformar2
   |
   v
persistir
```

A execução manual terminou com status de sucesso e todas as quatro tasks foram concluídas sem falhas.

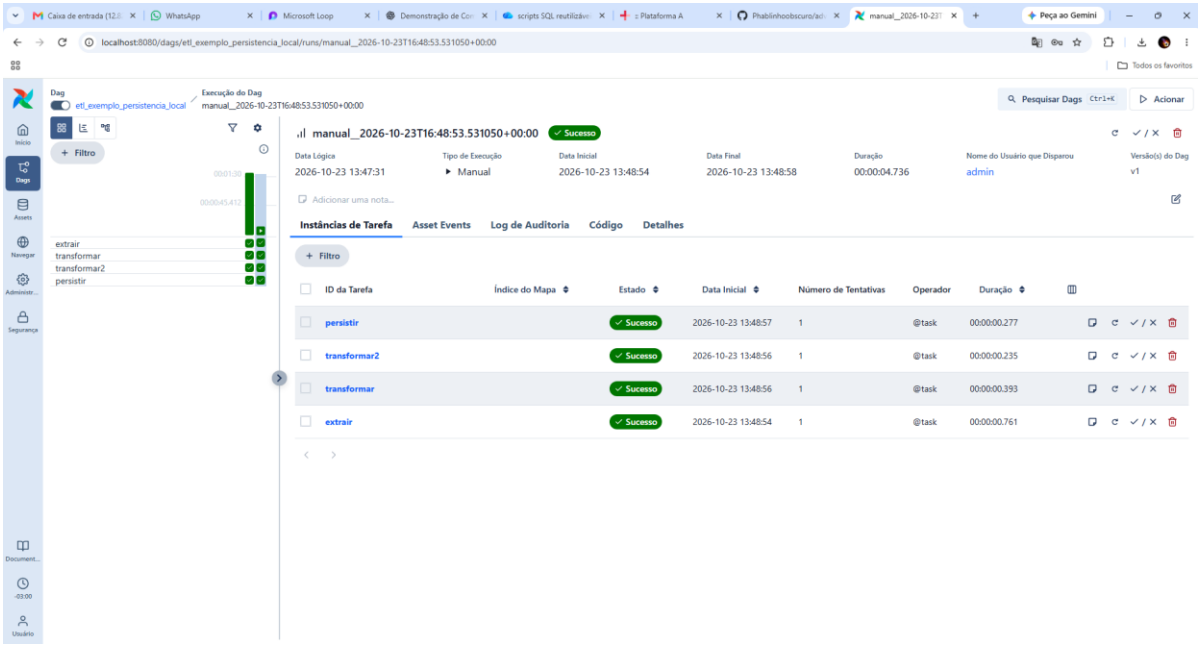


Figura 5 - Execução bem-sucedida da DAG etl_exemplo_persistencia_local.

8. Persistência de dados

A task `persistir` gerou o arquivo `data/resultado_etl.json` no volume local mapeado pelo Docker. O arquivo confirmou que a DAG conseguiu extrair, transformar e persistir os dados.

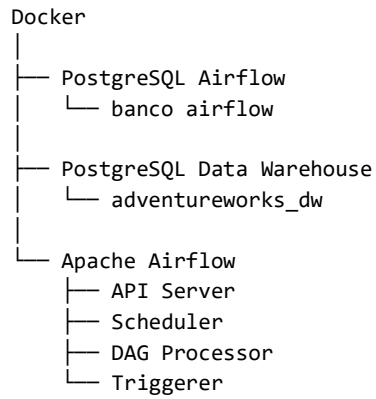
```
[
  {
    "id": 1,
    "nome": "Produto A",
    "quantidade": 3,
    "valor_total": 30
  },
  {
    "id": 2,
    "nome": "Produto B",
    "quantidade": 7,
    "valor_total": 70
  }
]
```

9. Classificação dos componentes

Componente	Decisão	Justificativa
Apache Airflow 3.3.1	Reutilizar	Base executada e validada.
PostgreSQL 16	Reutilizar	Será mantido para metadados do Airflow.

LocalExecutor	Reutilizar	Adequado ao ambiente acadêmico local.
Estrutura Docker	Reutilizar como base	Infraestrutura funcional e já testada.
docker-compose.yml	Adaptar	Deverá receber um PostgreSQL separado para o Data Warehouse.
.env.example	Adaptar	Deverá incorporar configurações específicas do AdventureWorks.
Padrão @dag / @task	Adaptar	Servirá como padrão de implementação da DAG do projeto.
SQLExecuteQueryOperator	Adaptar	Pode ser usado para executar scripts SQL do DW.
etl_exemplo.py	Referência	Exemplo didático de tasks e dependências.
etl_build_schema_dw.py	Referência conceitual	Exemplo útil de execução de SQL pelo Airflow.
etl_build_schema_stg.py	Referência	Staging não é requisito obrigatório do projeto.
extract_blockbuster.py	Referência	Código específico do domínio de exemplo.
dags/sql/*.sql	Não copiar	SQLs pertencem ao domínio Blockbuster.

10. Arquitetura pretendida após adaptação



11. Conclusão

O material disponibilizado pelo professor foi validado com sucesso em sua configuração original. A infraestrutura Docker foi inicializada, o Apache Airflow tornou-se acessível pela interface web, os principais componentes ficaram operacionais e uma DAG completa foi executada com persistência local.

Dessa forma, a base pode ser reutilizada como infraestrutura do projeto individual. O trabalho futuro consistirá em adaptar o Docker Compose e as configurações para incorporar o Data Warehouse AdventureWorks, além de integrar a ETL incremental proveniente do projeto anterior.